From Weak Programs to a Router

A follow-up to `IMPROVEMENT_PLAN.md`. It tells the story in three parts: (1) how we found the weak judge programs and decided what to do, (2) how we acted on those directions, and (3) the router we learned - with all the accuracy numbers (routing measured at a **25% escalation budget**) and the escalation plot. The test-CV router is reported **separately at the end** because it is a ceiling, not a real metric.

All numbers are on the **721-row held-out test set**, scored against the LLM judge's labels. The committee's base accuracy (no routing) is **0.749**.

---

1. Finding the weak programs, and what to do next

We scored all 80 programs individually against both label sets (the 721 LLM-labeled rows and the 35 human-labeled rows). About 15 of them added little or no signal. Ranked by how well a program's score separates good from bad matches on the LLM set (AUC; 0.50 = no better than a coin flip), the **10 weakest** were:

program	rubric	test AUC
judge_77	Modifier/Qualifier	0.498
judge_34	Specificity	0.526
judge_61	Processing/Additive	0.528
judge_28	Nutrition proximity	0.541
judge_37	Specificity	0.544
judge_58	Processing/Additive	0.567
judge_74	Modifier/Qualifier	0.580
judge_62	Processing/Additive	0.585
judge_49	Food category	0.587
judge_52	Food category	0.592

They clustered in a few rubrics (specificity, modifier, processing, food-category), and failed for three plain reasons: they **assumed the two foods already matched** and only looked for conflicting words (so they scored almost everything high); they **matched loose single words** ("with", "in") out of context; and some **ranked fine but output on a squashed scale**, so the single yes/no cutoff threw their signal away.

The directions we decided to pursue (from `IMPROVEMENT_PLAN.md`): de-noise the committee, calibrate the aggregator, **learn a router** to escalate only the uncertain rows, repair the over-strict programs, and grow the human-labeled set.

---

2. Acting on those directions

- **Abstention (make weak programs stay silent).** We let each program abstain when its score is near its decision threshold. **Not a clean win** - it destabilized committee selection and gave no reliable

accuracy gain.
- **De-noise.** Pruning the weakest programs (by a validation-only criterion) left accuracy unchanged - they weren't being selected anyway. But replacing the vote-based aggregator with a **calibrated continuous aggregator** rescued the "posterior" confidence signal (see below).
- **Routing with the paper's signals.** Escalating the least-confident rows to the LLM works. **Vote disagreement** is a usable signal; the **raw Snorkel posterior was poorly calibrated** and barely beat random - which the calibrated aggregator fixes.
- **Learn a router.** The main new step - covered next.

---

3. Learning a router

What it is. A small classifier that looks at a row and predicts **"will the committee get this one wrong?"** We then spend our LLM budget on the rows it flags as most-likely-wrong.

- **Input:** only committee outputs for the row - every program's score, summary stats, the aggregator's confidence, vote disagreement, abstain counts, and simple pair features (word overlap, a "hard category" flag). **No LLM call.**
- **Output:** a number = probability the committee is wrong (used to rank rows).
- **Ground truth to train on:** `committee_wrong = 1 if committee's answer != LLM's answer`. We generate this honestly on the validation set using 5-fold out-of-fold refits.
- **Training data:** the **222 validation rows** - of which only **22 were committee errors** (a small, imbalanced training signal; this matters below).
- **How the budget enters:** the router only *ranks* rows. At serving time we escalate the top **X%** most-likely-wrong rows (here **X = 25%**); the rest keep the committee's answer.

How much data each piece was trained on

The whole system is deliberately data-light. Here is exactly what was trained on what:

Component	Trained on	Rows	Positive (error) examples	Features
The 80 judge programs (synthesis)	10 few-shot from val	10	-	-
Committee fit (thresholds, top-k, Snorkel)	val	222	-	-
Continuous aggregator (a router feature)	val	222	-	-
Val-trained router (deployable)	val (OOF targets)	222	22 errors	85 (basic)
Test-CV router (ceiling)	test, in 5-fold CV	~577 per fold (721 total)	181 errors (~145/fold)	~97 (enriched)

Key points on the router training specifically:

- **What matters is the number of *error* examples, not rows.** The router learns to spot the committee's mistakes, so its training signal is the positives (errors). The deployable router saw only **22 errors** in

222 rows; the ceiling saw **~145 per fold**. That ~7x difference in error examples - not the model or its hyperparameters - is what separates ~0.86 from ~0.95.

- **The 22 errors are also unrepresentative.** The validation set has a 9.9% error rate versus 25% on test, because the programs were synthesized from validation few-shot, making validation artificially easy. So the deployable router trains on both *too few* and *too easy* errors.
- **This is why tuning didn't help** (XGBoost reached 0.95 cross-validation AUC on val yet still needed ~47% escalation on test): you cannot tune your way out of a non-representative, 22-example training signal.
- **The fix is data, not modeling:** a few hundred representative, labeled error examples (disjoint from the synthesis few-shot) would let the deployable router approach the ceiling.

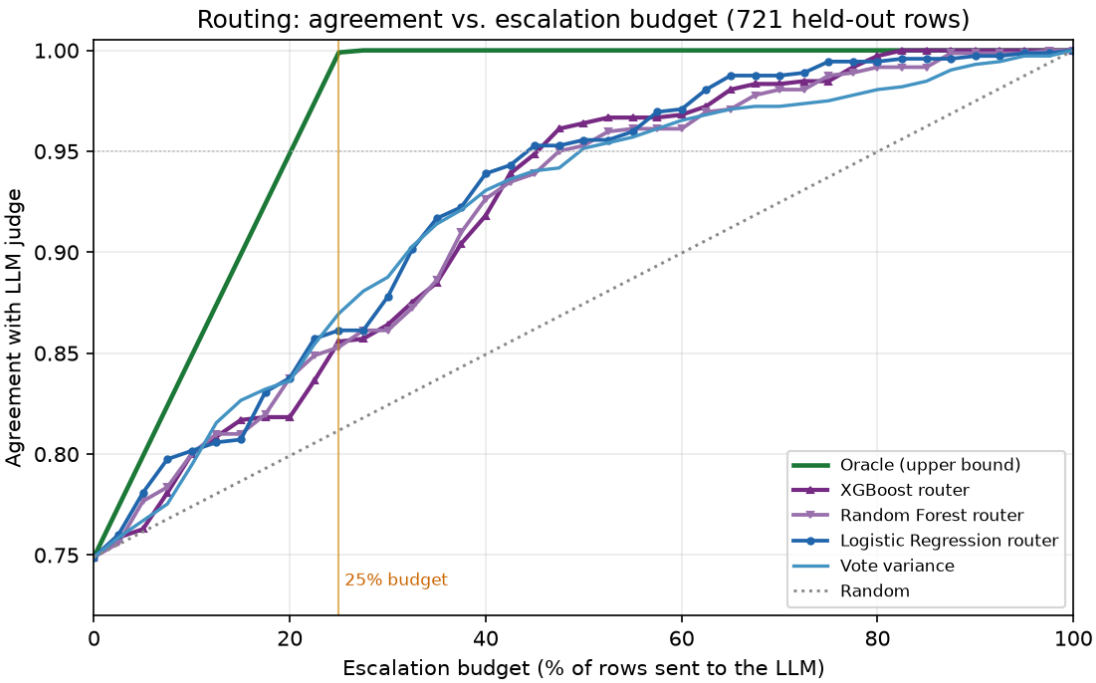
Accuracy of every experiment (routing at 25% escalation)

Group	Method	Accuracy @25%
Base (no routing)	Programmatic committee	0.749
Base (no routing)	Continuous calibrated aggregator	0.750
Routing - baselines	**Oracle** (best possible)	0.999
Routing - baselines	Random	0.812
Routing - paper signals	Vote variance	0.870
Routing - paper signals	Snorkel posterior margin	0.853
Routing - paper signals	Continuous-aggregator posterior	0.854
Routing - learned (deployable)	Logistic Regression	0.861
Routing - learned (deployable)	Random Forest	0.854
Routing - learned (deployable)	XGBoost	0.857

Abstention is on a different axis (it drops weak rows instead of escalating), shown here with its coverage, not comparable to the 25%-budget rows above:

band	coverage	accuracy (committed rows)
0.00	1.000	0.749
0.05	0.863	0.794
0.10	1.000	0.723
0.15	0.943	0.740

Escalation plot



Reading it: at a 25% budget the deployable routers and paper signals all land around **0.85-0.87**, well above random (0.81) but well below the oracle (~1.0). The learned routers only marginally beat the simple vote-variance signal - because the router had just **22 error examples** to learn from, drawn from an easy validation set (the programs were built using validation examples, so validation is not representative of test). **The limit is the training data, not the model** - hyperparameter tuning of LR and XGBoost changed nothing.

---

4. The test-CV router - a ceiling, NOT a real result

To ask *"how good could the router be if it had representative training data?"*, we also trained it by cross-validation **on the test set itself** (with the richer feature set). This gives:

Method	Accuracy @25%
XGBoost (test-CV, enriched)	**0.949**
Logistic Regression (test-CV, enriched)	0.935

These are not deployable numbers and must not be quoted as results. They use the **test labels to train the router**, which you would not have in production. It is a legitimate *ceiling estimate* (the same way the PAJAMA paper evaluates its ModernBERT router), showing that **with representative labeled data the router could get close to the oracle (~0.95 at 25%)** - but the honest, deployable number today is the validation-trained **~0.86 at 25%**. The gap between 0.86 and 0.95 is exactly the value of collecting a representative, labeled router-training set.

---

5. Also tried: text representations of the two foods

So far the routers only see the committee's program outputs, not the actual words of the two foods. We added **text representations of item 1 (ingredient) and item 2 (USDA match)** as extra router features, in two flavors - both reduced to the same 37-feature layout (each string -> 12 dims, plus their absolute

difference (12) and cosine similarity (1)), and both fit unsupervised on **validation text only**, so neither adds any leakage:

- **TF-IDF** - character n-gram TF-IDF of each string, reduced with SVD. Deterministic, local, no model.
- **BGE-m3** - the backend's `local` neural embedding model (`BAAI/bge-m3`, 1024-dim) embeds each food name; we reduce with SVD but keep the cosine from the **full 1024-dim** vectors (the real semantic signal). Local model, no API call.

Accuracy @25% escalation / escalation needed to reach 0.95 (one consistent run, so the baselines match across columns):

	no text	**+ TF-IDF**	**+ BGE-m3**
Deployable (val-trained)	acc@25% / esc@0.95	acc@25% / esc@0.95	acc@25% / esc@0.95
Logistic Regression	0.861 / 45.0%	0.872 / 50.0%	**0.879 / 45.0%**
Random Forest	0.854 / 47.5%	0.856 / 50.0%	0.859 / 50.0%
XGBoost	0.857 / 45.0%	0.836 / 52.5%	0.849 / 50.0%
Ceiling (test-CV, XGBoost)	0.947 / 26.7%	**0.954 / 25.8%**	0.947 / 27.5%

What it showed: - **BGE is the best deployable text addition.** It lifts Logistic Regression to **0.879** - the best deployable number in the whole study - and, unlike TF-IDF, it does *not* raise the escalation needed for 0.95 (stays at 45%). BGE's semantic cosine carries real food-meaning the linear model can use. It also nudges Random Forest up slightly (best RF too). - **Only the linear model reliably benefits.** XGBoost gets *worse* with either text source - the familiar **overfitting** of trees when you add 37 features to a 222-row / 22-error set. - **At the data-rich ceiling the story flips:** it's char-level **TF-IDF** (0.947 -> 0.954), not BGE (no change), that gives the small win - once there are enough labeled errors, the exact- string overlap TF-IDF captures helps more than BGE's semantics, which the committee features already largely encode.

Takeaway: of everything tried, **BGE embeddings are the one text feature worth adopting in the deployable router - but only for Logistic Regression**, where they buy a real ~1.8-point gain at no extra escalation and reuse the backend's existing local model (no new API). For the tree models and the ceiling, text remains marginal. The headline lesson is unchanged: the ~0.86 -> 0.95 gap closes with **representative labeled errors**, not richer inputs.

6. Also tried: a neural router (DistilBERT)

The final experiment swaps the feature-based router for a fine-tuned **hybrid DistilBERT**: `distilbert-base-uncased` reads the two food names (`"ingredient: X [SEP] usda match: Y"`), its `[CLS]` vector is concatenated with the **15 committee signals** (aggregator posterior & margin, continuous-aggregator prob & margin, vote variance/entropy, vote counts, and the score summary stats), and a small head predicts `committee_wrong`. Same two settings and target as every other router.

setting	**acc@25%**	**esc for 0.95**	**wrong-AUC**
Deployable (val-trained, 22 errors)	0.829	77.5%	0.608
Ceiling (test-CV, ~145 errors/fold)	0.938	30.0%	0.909

For reference: best deployable feature-router is **LR+BGE 0.879 / 45%**; the XGBoost ceiling is **0.947-0.954 / ~26%**; random routing is 0.812.

What it showed: - **Deployable: the worst router in the study (0.829)** - barely above random (0.812), well below the simple LR (0.861), with a wrong-AUC of **0.608** (0.5 = coin flip) and needing to escalate **77.5%** of rows to reach 0.95. A 66M-parameter transformer cannot learn from **22 error examples**; it overfits and its P(wrong) ranking is close to noise. The bigger the model, the more representative data it needs - and here we have the least. - **Ceiling: strong (0.938, AUC 0.909) but still below the XGBoost ceiling** and needing more escalation (30% vs ~26%). Even with representative labels, the fine-tuned text encoder does not beat gradient-boosted trees on the committee features - the committee already consumed the food-name text when it ran the programs, so the committee signals carry most of what a router can know.

Takeaway: a neural router is **not warranted here**. It loses badly when data is scarce (the real, deployable regime) and doesn't win even when data is plentiful. The practical router remains **Logistic Regression (optionally + BGE features)**; the lever is still representative labeled errors, not a bigger model.

7. Cost analysis at a 25% escalation budget

Framed the way the PAJAMA paper frames it: program synthesis is a one-time **O(1)** investment, the committee makes **zero** per-sample API calls (80 Python programs run locally), and routing sends only a fraction of rows to the LLM. At a 25% budget that fraction is **0.25·N**, i.e. **4x fewer LLM calls** than grading everything.

Assumptions (swap in exact rates as needed). LLM judge = `openai/gpt-oss-120b` on Groq, temp 0. From the actual judging prompt: ~320 input tokens (fixed rubric + the two names) + ~300 output tokens (it reasons step-by-step in JSON) ~ 620 tokens/call. At Groq list price (~$0.15/1M in, ~$0.75/1M out) that is **~$0.00027/call**. A **frontier judge** (GPT/Claude in the loop, ~$2.50/1M in, ~$10/1M out) costs **~$0.0038/call**. One-time synthesis of 80 programs is **~$6** (the paper measured $6.49-$7.21), paid once and reused forever. "Accuracy" here = agreement with the LLM judge (the distillation target), so LLM-only = 1.000 by identity and routed rows are correct by identity.

Per-sample cost (routed @25% = 0.25·N LLM calls):

Workload N	LLM-only calls	Routed @25% calls	Groq gpt-oss-120b: all -> @25%	Frontier judge: all -> @25%
943 (our test+val)	943	236	$0.26 -> **$0.06**	$3.6 -> **$0.9**
10,000	10,000	2,500	$2.73 -> **$0.68**	$38 -> **$9.5**
100,000	100,000	25,000	$27 -> **$6.8**	$380 -> **$95**
1,000,000	1,000,000	250,000	$273 -> **$68**	$3,800 -> **$950**

Plus the one-time ~$6 synthesis, which amortizes to ~$0/sample at scale.

What the 4x cut in LLM calls buys:

Config	LLM calls	Agreement w/ LLM judge	Throughput vs LLM-only
LLM-only (grade everything)	N	1.000 (by definition)	1x

Config	LLM calls	Agreement w/ LLM judge	Throughput vs LLM-only
Committee only (no routing)	0	0.749	~hundredsx (local)
PAJAMA + routing @25%	**0.25·N**	**0.861 (LR) - 0.879 (LR+BGE)**	**~4x**
Oracle router @25% (ceiling)	0.25·N	~0.999	~4x

Reading it. - **75% fewer LLM calls** (4x) at a flat 75% cut on the per-sample bill at *every* scale - the ratio is pricing-independent. Dollars are small for a cheap Groq open-weight judge, but the same lever on a **frontier judge is $3,800 -> $950 per million rows**, which is where the cost story lands for a production deployment. - That 25% budget **recovers ~45-52% of the committee->LLM accuracy gap** (0.749 -> ~0.86-0.88). The **oracle reaches ~1.0 at the same 25%**, because the committee's error rate (~25%) almost exactly equals the budget - so a perfect router would need to escalate essentially only the wrong rows. The gap between our ~0.88 and that ~1.0 is exactly the "improve aggregate performance" headroom. - **Throughput ~ 4x** vs LLM-only (75% of rows get instant local verdicts), consistent with the paper's 2.2-2.9x at a *higher* 32-42% fallback.

8. Bottom line

- The weak programs are real but pruning them didn't help; the two things that *did* help are **calibrated aggregation** (fixes the confidence signal) and **routing** (escalate the uncertain rows).
- **Deployable today:** escalate 25% of rows -> **~0.86-0.88 agreement** with the LLM (~4x fewer LLM calls than grading everything). The best deployable router is **Logistic Regression with BGE-m3 text features (0.879)**; the next-best signals are the plain learned router and the calibrated posterior (~0.86).
- **Model choice is settled:** across linear, tree, and neural routers, the simple **Logistic Regression** wins the deployable regime and a fine-tuned **DistilBERT is the worst** (it needs far more data than 22 errors). Even at the data-rich ceiling, XGBoost on committee features edges out DistilBERT. Bigger models do not help.
- **Biggest lever next:** give the router **representative, labeled training data** (disjoint from the program-synthesis examples). That is what moves it from ~0.86 toward the ~0.95 ceiling - not more model tuning, and not a bigger model.